

Temporal Referent Integrity: A Controlled Diagnostic of Referential Resolution Timing in Tool-Using Agents

Anonymous submission

Abstract

Stable-only and one-sided evaluations can conflate a referent already committed to an entity with a selector deliberately deferred to a later world state. We introduce *temporal referent integrity* (TRI), a controlled diagnostic for selective referent re-resolution in tool-using agents. Matched Preserve/Reevaluate pairs hold states, selector, action, and transition fixed while assigning opposite gold targets; changed-winner pair accuracy (PairAcc) therefore rejects both Always-Lock and Always-Reevaluate policies. Across a 240-task cross-schema replication, Generic controllers replace a correctly bound, still-actionable target with the refreshed selector winner in 43/72, 38/80, and 59/79 eligible cases for Qwen, GLM, and DeepSeek. A fixed-executor consistency check maps every such substitution to a wrong-entity write, whereas matched Compile-then-act records none. Under equal calls and actor payloads, exposing a resolution-timing decision raises changed-winner PairAcc for Qwen and GLM on the authored diagnostic. Transfer is model-dependent, with a clear end-to-end gain only for GLM on source-derived pairs. Author-built public-suite retrieval finds no strict native opportunity, but recall is uncalibrated. A post-hoc event-order rule performs strongly on authored inventories but transfers poorly. TRI currently covers controlled, single-refresh scalar workflows; native-workflow prevalence and open-language coverage remain unresolved.

Introduction

Consider two requests issued before the same mailbox refresh:

- (1) “Choose the highest-priority unread email now. Refresh the mailbox, then reply to it.”
- (2) “Refresh the mailbox first. Then choose the highest-priority unread email and reply.”

Suppose email A is highest priority before refresh and email B afterward. The environment transition is identical, but the correct target differs. Request (1) establishes A as

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

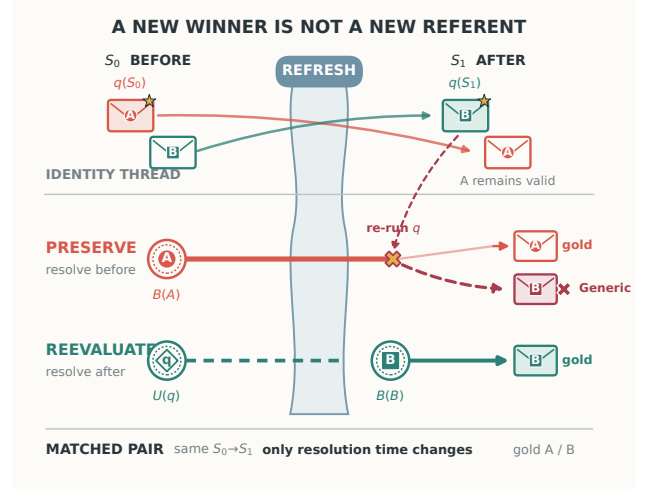


Figure 1: Matched opposite-gold logic and an observed Cross-Schema Replication consequence. The same $S_0 \rightarrow S_1$ transition requires the old target under Preserve and the refreshed winner under Reevaluate, so either unconditional policy fails one member. In the shown Preserve run, Qwen Generic stored the old entity identifier but wrote to the refreshed winner.

an identity commitment; request (2) deliberately leaves a description to be evaluated in the new state. Acting on B in request (1) replaces a committed referent and produces a wrong-entity action.

A transcript can contain both the original target and the refreshed state, yet the final actor may apply the old selector again in the new state. Unconditionally retaining the old entity identifier (ID) causes the complementary error on Reevaluate requests. Correct behavior depends on when the instruction commits the action referent.

TRI separates three variables at the refresh boundary: world evidence and the current selector winner; referential control state, which records a committed entity or a deferred selector; and action validity, which checks whether a candidate supports the operation. A refresh can change evidence

without rewriting a commitment. Validity can reject a candidate but cannot select a discourse-licensed replacement. Here “authorization” refers only to resolution timing.

Evaluations built from Stable or one-sided cases can conflate a referent already committed to an entity with a selector deliberately deferred to a later world state. TRI makes this distinction observable through matched opposite-gold pairs that separate selective re-resolution from unconditional retention and recomputation.

The paper makes three contributions:

- We formalize bound and deferred resolution as distinct referential control states, separate from world evidence and action validity, and define the observations needed to attribute post-refresh target substitution.
- We construct matched Preserve/Reevaluate tasks with identical states, selectors, actions, and transitions. Changed-winner PairAcc distinguishes selective re-resolution from Always-Lock and Always-Reevaluate within this controlled policy class.
- We trace correctly bound targets through substitution, deterministic target-to-write checks, and model-facing database mutations. Equal-call interventions and transfer tests compare several operationalizations of an explicit resolution-timing decision, with model-dependent results.

The experiments follow four questions: whether the diagnostic separates referential policies, whether substitution occurs after correct binding, whether it changes the executed entity, and whether an explicit timing decision helps under matched calls and new expressions.

Related Work

Reference, binding, and memory. Discourse theories distinguish referents from changing descriptions and salience (Kamp and Reyle 1993; Grosz, Joshi, and Weinstein 1995); coreference work recovers textual identity links (Lee et al. 2017; Joshi et al. 2020; Dasigi et al. 2019). Babu and Indukuri (2026b) study failures in initial tool-argument binding; TRI conditions on correct $bind(r, S_0)$ and observes later target use. Agent memory and entity tracking retain facts, IDs, and traces (Tang et al. 2026; Packer et al. 2023; Sumers et al. 2024). TRI adds the control-state question of whether a retained selector output denotes a committed referent. Temporal Blindness studies when time should trigger a new observation (Cheng et al. 2026), prospective-memory evaluation tests delayed intentions (Liu and Gabriel 2026), and evolving-environment benchmarks measure belief and memory adaptation (Ji et al. 2026; Xu et al. 2026). Our diagnostic isolates the propagation boundary between updated world evidence and an action referent.

Post-binding persistence and resolution timing. Concurrent Binding Drift work studies whether a correctly bound primary carry slot remains stable, how an initially wrong binding propagates, and whether entity locks or large language model (LLM) re-verification reduce these errors (Babu and Indukuri 2026a). Its workflows may also contain derived

Problem axis	What it establishes	What remains unresolved
Initial binding	Correct target in S_0	Whether later replacement is licensed
Entity memory	Old and new IDs/states retained	Which identity the action should consume
Temporal reasoning	Event and update order recovered	Whether the reference was already resolved
Binding drift	A carry-slot target changed later	Whether a description was deliberately deferred
TRI matched pair	Bound versus deferred status	Selective re-resolution under one transition

Table 1: Problem boundary. Binding Drift evaluates persistence of a committed primary carry slot (Babu and Indukuri 2026a); TRI varies whether the action referent is committed or deferred and scores opposite targets under one transition.

roles; the defining endpoint is persistence of an already committed primary referent. TRI varies the control state of the same action-target description under the same transition. The Preserve member commits to $e = bind(r, S_0)$ before refresh, while the Reevaluate member defers the referent and assigns gold $q(S_1) \neq q(S_0)$. Binding Drift characterizes stability after commitment. TRI identifies whether commitment preceded the transition. These are complementary axes, summarized in Table 1.

Tool agents and executable constraints. Tool-use benchmarks cover application programming interfaces (APIs), web environments, and stateful interaction (Schick et al. 2023; Li et al. 2023; Qin et al. 2024; Patil et al. 2023; Liu et al. 2024; Lu et al. 2025; Trivedi et al. 2024; Yao et al. 2024; Barres et al. 2025; Sierra Research 2026; Ruan et al. 2024). Process-aware graphical-interface evaluation likewise shows that final-state success can hide a critical intermediate error (Yang et al. 2026). The pinned suites audited here lack gold labels for strict post-binding Preserve/Reevaluate opportunities; broader tool-use benchmarks commonly emphasize task completion. LedgerAgent and Bounded Autonomy add typed state or action contracts (Uddin et al. 2026; Sohail and Haider 2026). Such validity constraints filter candidates, while discourse supplies the intended identity. AgentSpec and policy autoformalization compile or enforce rules, VIGIL monitors temporal and value-flow constraints, and Cordon stages multi-step effects before commit (Wang, Poskitt, and Sun 2025; Mondl, Maisel, and Brock 2026; Li et al. 2026; Chen et al. 2026). These systems address executable constraints; our matched design evaluates the bound-versus-deferred resolution contrast. Structured generation can enforce syntax (Beurer-Kellner, Fischer, and Vechev 2023; Willard and Louf 2023; Khattab et al. 2024), while our clustered minimal pairs follow diagnostic and contrast-set methodology (Ribeiro et al. 2020; Gardner et al. 2020). Meta-evaluation studies discrimination across model capabilities (Wang et al. 2026); TRI asks whether a regime discriminates referential policies that can obtain the same score. Complementary semantic conditions have similarly exposed one-sided large

language model (LLM) heuristics and correction trade-offs in aspectual natural-language inference (NLI) (Ma and Miyao 2026); TRI applies the diagnostic logic to post-binding control and executed actions, not lexical aspect or static entailment.

Temporal Referent Integrity and Evaluation Identifiability

Let H be the instruction and interaction history before a refresh, S_0 and S_1 the states before and after it, r an action-target reference, and $q_r : S \rightarrow E \cup \{\perp\}$ its deterministic task resolver. For each frozen task, $q_r(S)$ returns one eligible entity, or $\perp$ when none is eligible; unresolved ties are excluded. The formalization keeps three variables distinct: S and $q_r(S)$ represent world evidence, $C_H(r)$ represents referential commitment, and $V_a(e, S)$ represents action validity. Let $C_H^-(r)$ and $C_H^+(r)$ denote control state immediately before and after refresh. History induces one of two pre-refresh states:

$$C_H^-(r) \in \{B(e_0), U(q_r)\}, \quad e_0 = q_r(S_0). \quad (1)$$

$B(e_0)$ is an identity commitment resolved before refresh; $U(q_r)$ is a query intentionally left unresolved until afterward. Updating world evidence can change $q_r(S)$ without changing a bound commitment:

$$S_0 \neq S_1 \not\Rightarrow C_H^+(r) \neq C_H^-(r). \quad (2)$$

Under Preserve, $C_H^+(r) = B(e_0)$. Under Reevaluate, the unresolved query $U(q_r)$ is evaluated only in S_1 , yielding $B(q_r(S_1))$. Replacing an existing $B(e)$ with a distinct $B(e')$ requires the additional discourse-sensitive decision $\Gamma_H(r, e \rightarrow e')$.

We represent each workflow with four observable events: OBSERVE, BIND, REFRESH, and WRITE. The target is resolved from an observed collection, a state transition occurs, and a later entity-directed mutation consumes the target. Public-benchmark coverage records which events and resolution-time contrasts are observable.

An identifying trace requires a binding, an update, a changed selector winner, a still-valid old target, and a later entity-directed action. The initial ID separates binding errors from later substitution. Matched Preserve/Reevaluate cases share one transition and require opposite targets, which tests both unconditional policies. A state diff or target-level execution record then connects the selected target to the resulting write. This yields three evidence levels: policy behavior, error attribution, and executed consequence.

Resolution timing and action validity. For execution, $V_a(e, S)$ conjoins the action-schema preconditions, and invalid-target policy $\pi : E \times S \rightarrow E \cup \{\perp\}$ returns $\perp$ or, when authorized, $q_r(S_1)$. The authorized target is

$$T^* = \begin{cases} e_0, & C_H^-(r) = B(e_0) \wedge V_a(e_0, S_1), \\ q_r(S_1), & C_H^-(r) = U(q_r), \\ \pi(e_0, S_1), & C_H^-(r) = B(e_0) \wedge \neg V_a(e_0, S_1). \end{cases} \quad (3)$$

The first two branches define the actionable TRI core. The third is a separate execution policy: when the committed

Evaluation regime	Lock	Reevaluate	Selective?
Stable only	can pass	can pass	No
Changed Preserve only	can pass	fails	No
Changed Reevaluate only	fails	can pass	No
Matched changed PairAcc	zero	zero	Yes

Table 2: Policy identifiability. Stable or one-sided evaluation can favor unconditional policies; matched changed-winner PairAcc rejects both extremes within the three-policy class.

target is invalid, a system may reject, clarify, or reselect if the instruction permits it. Referential identity and fallback choice are therefore scored separately. Controllers operationalize the timing decision in different ways; one representation used below is $L = (m, e, q, V_a, \pi)$. Action validity accepts or rejects a candidate but supplies no replacement identity.

Benchmark-design observation. Consider a Preserve/Reevaluate minimal pair with identical S_0 , S_1 , q , and a , where $e = q(S_0)$, $e' = q(S_1)$, and $e \neq e'$. The authorized targets are e and e' , respectively. For exact-target observations within the policy class {Always-Lock, Always-Reevaluate, Selective}, discrimination requires one changed Preserve case and one changed Reevaluate case. Either row alone leaves Selective observationally equivalent to one unconditional policy. The opposite-gold pair separates all three and is the smallest such set in this restricted class. This is a benchmark-design condition, not a general result for arbitrary stochastic controllers; the short argument is supplementary.

More generally, an evaluation can identify a policy only when its scored tasks separate that policy from relevant alternatives. Stable tasks make lock and reevaluation observationally equivalent, while a one-sided changed-winner test can reward one unconditional extreme. The crossed matched regime breaks both equivalences. For a matched pair, PairAcc is one only when both members are correct (Table 2).

We separately report exact target/final state, wrong-entity writes, invalid attempts, and unnecessary rejection; always rejecting avoids writes but fails every actionable task.

Diagnostic Policies and Controller Probes

The Generic Structured Ledger controller (Generic) retains the instruction and selected ID without encoding resolution mode. The comparison controllers are mechanism probes for explicit resolution-timing decisions. Compile-then-act (CTA) emits binding time, selector, and a bound ID before refresh without a deterministic gate. Lifecycle records the same timing decision together with validity and fallback fields; its actor-mediated and gated variants test soft and hard use of that record. Both compilers use discourse order: “identify, refresh, act on it” preserves a resolved identity, while “refresh before deciding” evaluates the selector in S_1 . The complete record schema and gate branches are supplementary.

The anonymous Code and Data Supplement contains prompts, runner interfaces, inventories, raw outputs, and executable reports. Controllers receive the instruction’s natural-language selector. Gold targets, normalized selector fields,

and pre-/post-refresh winner IDs are withheld.

For information parity, compilers see the instruction, S_0 , action schema, and planned refresh. Generic retains the instruction, selected ID, snapshot, natural-language selector, action, and preconditions for its second-call actor. CTA, Lifecycle-Actor, and Generic each use two calls; Lifecycle-Gated skips the actor for a valid Preserve branch, making the primary package comparison call-asymmetric. Complete settings are in the Code and Data Supplement. These probes test operationalizations of the same timing variable. The evidence supports no unique requirement for lifecycle typing, pre-refresh compilation, or hard gating.

Experimental Setup

We evaluate the resolution-time distinction using the Matched Timing Diagnostic, Cross-Schema Replication, and supplementary Action-Validity Stress Test.

Controlled task construction. Each domain specification defines an entity table, a natural-language selector, its ranking or Boolean criterion, an action, and explicit action preconditions. The generator first evaluates the selector on S_0 to obtain e_0 , applies a controlled transition, and evaluates it on S_1 to obtain e_1 . A language template determines whether the action target is e_0 , e_1 , $\perp$, or a conditionally guarded choice. Paired tasks share the domain, both states, selector, action, and schema; only discourse order and anaphoric language change. This prevents domain difficulty or target frequency from explaining the preserve/reevaluate contrast.

Workflow task generation. The generator organizes selector questions into a four-event workflow: OBSERVE a user-visible collection, BIND one stable ID, apply an exogenous or synchronization-style REFRESH, and execute a target-specific WRITE. Domain templates fill these slots with ordinary operations (reply to a message, comment on a reminder, update a project or expense, change an inventory record, or deploy a selected artifact). The same-role replacement requirement is added when the application exposes both a stable ID and a ranking or eligibility selector. This yields a crossed factorial 2×2 core (Preserve/Reevaluate $\times$ Stable/changed-winner), while Remove, Invalidate, and name collision are orthogonal stressors. The coverage audit records which parts of this structure appear in public suites; deployment frequency requires natural workflow data.

The crossed core combines Preserve/Reevaluate with an unchanged or changed selector winner. *Stable* controls expose overreaction to refresh; *changed-winner* and *name collision* create opposite-target pairs while retaining valid stable IDs. *Remove* and *Invalidate* form a separate fallback-policy slice; the complete transition taxonomy is supplementary.

Blind human construct validation. One adult volunteer produced natural-language rewrites of 50 sampled scalar instructions; three other adults, uninvolved in TRI design, independently labeled 100 randomized original/rewrite items using opaque IDs without condition metadata or gold. All four volunteers provided informed consent; no sensitive personal information was collected, released responses

Evidence unit (status)	Denominator	Measured scope
Package runs: Qwen primary/frozen; GLM post-primary	160 tasks/model; 20 clusters: 128 actionable + 32 Reject	End-to-end package contrast; components analyzed separately
Changed pairs (post-primary audit)	Matched Timing Diagnostic: 32 pairs; expanded audit: 80 pairs	Selective PairAcc in controlled pairs
Cross-Schema Replication (post-primary)	240 rows = 120 pairs; shared eligible 66/70/69	Schema/state transfer with authored language
Full matched-call confirmation (post-primary)	160 rows = 80 pairs/model; 32 actionable changed pairs	Decision visibility under equal calls and base payloads
Volunteer-rewrite transfer (post-primary)	50 rows/model; 40 actionable; 48 determinate labels; 3 changed pairs	One-volunteer rewrites of authored tasks; not open-language validation
Source-derived matched-call contrast (post-primary)	60 rows/model = 30 changed pairs; 10 pairs/source	Controlled interventions over source states and schemas
External audits (post-primary/descriptive)	Six public suites; 320 source-transfer attempts	Coverage, retrieval, and lower-intervention behavior
Post-hoc event-order rule	Three authored inventories	Simple event-order extraction on authored language

Table 3: Evidence map and denominators. Full chronology, controller-specific eligible counts, intervals, and failure accounting are supplementary. Planned or unverified analyses are not results.

were de-identified, compensation, where provided, was non-contingent, and no formal IRB or exemption determination was obtained. We report agreement, majority-gold alignment, and consensus sensitivity only for the single-target scalar core. Referent identity is the TRI construct; whether an invalid action should Reject, Clarify, or select another entity is a separate execution-policy judgment.

The frozen error taxonomy separates temporal rebinding, premature binding, invalid processing, unnecessary rejection, and selector grounding; definitions are supplementary.

Direct-neighbor baseline. Generic Structured Ledger stores the initial ID, entity snapshot, selector, action, and preconditions but omits reference mode and fallback semantics; its actor still receives the original instruction and refreshed state. Frozen CTA is rerun unchanged as a neighboring pre-refresh commitment baseline. The full component comparison, including the untyped planner, appears in Supplementary Material and compares implementations of the timing decision on scalar tasks.

For a direct policy contrast, we also evaluate two deterministic extremes without model calls. Always-Lock + validity retains the S_0 ID and rejects it when the refreshed action preconditions fail; Always-Reevaluate discards the binding and applies q to S_1 . These are deterministic realizations of

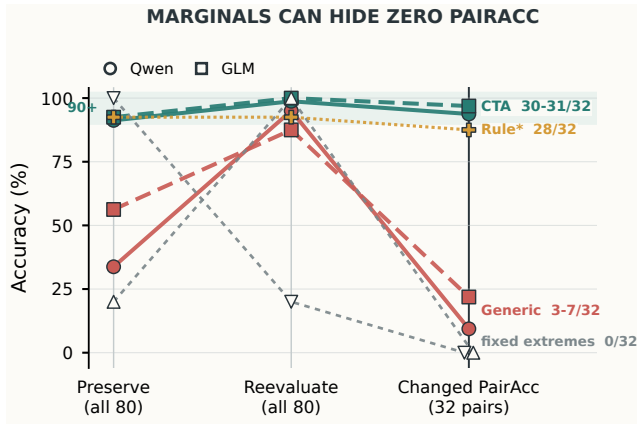


Figure 2: Policy marginals and matched discrimination on the Matched Timing Diagnostic. Each path joins Preserve and Reevaluate accuracy over 80 pairs to changed-winner PairAcc over 32 matched pairs; segments connect metrics, not time. Circles and squares identify Qwen and GLM. The two unconditional fixed policies each reach 100% on one marginal but 0/32 PairAcc; Generic obtains 3–7/32, CTA 30–31/32, and the post-hoc Rule* 28/32. Package runs mix Qwen primary/frozen and GLM post-primary evidence; changed-pair metrics are post-primary audits.

the two unconditional extremes. On the frozen 160-task inventory both reach 96/160 (60.0%), but their mode slices are complementary: Always-Lock reaches 80/80 anchored and 16/80 dynamic, whereas Always-Reevaluate reaches 16/80 anchored and 80/80 dynamic. Their complementary slices expose the selective re-resolution requirement.

Model-facing SQLite trajectories. The 40-task SQLite subset covers all templates, domains, modes, and updates. An action-validating mutation tool records final state, wrong writes, invalid attempts, and collateral changes. This controlled execution test connects selected targets to database mutations.

Models, inference, and auditing. Qwen3.5-122B-A10B (Qwen) and GLM-5.1 (GLM) share a temperature-zero OpenAI-compatible protocol; a post-primary DeepSeek-V4-Pro (DeepSeek) run adds a third family. Smoke and failure accounting are supplementary. Task accuracy is intention-to-treat (ITT). Conditional substitution additionally requires a correct observable initial ID, completed refresh, surviving/action-valid old target, and distinct refreshed winner. The anonymous artifact includes complete metric definitions, frozen data, protocols, raw outputs, traces, hashes, and source-derived reports.

Results

Diagnostic Discrimination

Always-Lock and Always-Reevaluate pass Stable cases and opposite one-sided changed-winner slices, yet both obtain zero PairAcc on complete changed pairs. A zero-API policy audit confirms the selection problem: all 15 maxi-

mizer sets under Stable-only or one-sided scoring contain a zero-PairAcc policy, with worst-case PairAcc regret of 100 points. Aggregate end-to-end (E2E) scoring selects a PairAcc-optimal candidate in all five candidate sets. Figure 2 shows how the matched score separates the unconditional policies from selective re-resolution. Thus PairAcc reveals joint policy behavior and symmetric errors in the current study; we do not observe a controller-ranking reversal under aggregate E2E scoring.

The actionable referential core contains 128 tasks. Generic reaches 95/128 (74.2%) for Qwen and 93/128 (72.7%) for GLM; CTA reaches 126/128 (98.4%) and 127/128 (99.2%), and Lifecycle-Gated reaches 125/128 (97.7%) and 128/128 (100.0%). The remaining 32 anchored remove/invalidate rows use an author-specified REJECT gold and form a separate fallback-policy slice. Generic, CTA, and Gated obtain 8/32 and 22/32, 26/32 and 27/32, and 32/32 for Qwen and GLM, respectively.

The construct check supports the same separation. Across 100 blinded original and rewritten items, Fleiss' $\kappa = .708$, Krippendorff's $\alpha = .709$, and majority-gold agreement is 86% (91.5% among 94 determinate items). Agreement falls to 55% majority-gold and 25% unanimity for action-invalid REJECT. Against 46 determinate original-item majorities, Generic/CTA reach 69.6/91.3% for Qwen and 73.9/89.1% for GLM. We base the referential claim on the actionable core and treat rejection, clarification, and authorized reselection as execution policy.

Substitution After Correct Binding

We next condition on a correct observable initial ID, a completed refresh, a distinct new selector winner, and an old target that remains present and action-valid. In the primary diagnostic, Qwen Generic substitutes on 15/16 anchored flips and 14/16 name collisions; GLM substitutes on 3/16 and 7/16. Stable controls show no such concentration.

The 240-task Cross-Schema Replication extends this test to ten schemas. Generic selects the refreshed winner in 43/72 Qwen, 38/80 GLM, and 59/79 DeepSeek opportunities. CTA records zero substitutions among 71, 70, and 70 eligible cases. On shared-eligible rows, where both controllers bind correctly, the comparisons are 41/66 versus 0/66, 30/70 versus 0/70, and 50/69 versus 0/69 (Figure 3). The conditioning isolates replacement of a correct commitment from initial binding and target-validity errors.

Target-to-Write Verification

A deterministic fixed-executor consistency check turns all 43, 38, and 59 Generic substitutions into wrong-entity writes. Their cluster-bootstrap intervals are [46.5, 72.4], [35.0, 61.3], and [62.5, 86.1] percent. Initial-ID, missing-ID, action-invalid, API, and parse failures are excluded from this conditional denominator; Stable errors and non-core writes remain separate. Figure 4 reports the complete decomposition.

The model-facing 40-task SQLite subset supplies separate tool-loop evidence. Generic reaches 27/40 final states with 13 wrong writes for Qwen and 26/40 with eight wrong writes for

CTA IMPROVES BOTH OUTCOMES FOR ALL THREE MODELS

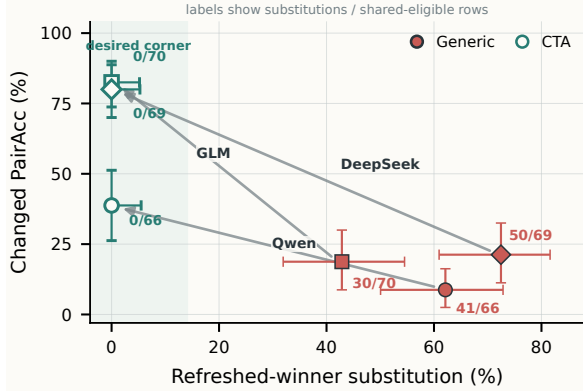


Figure 3: Cross-schema paired controller transitions on shared-eligible Preserve rows. The horizontal axis is conditional refreshed-winner substitution with Wilson 95% intervals; the vertical axis is PairAcc with cluster-bootstrap 95% intervals. Arrows connect each model’s Generic and CTA endpoints, not time, and labels report substitutions over the shared-eligible denominator. All three models move toward lower substitution and higher PairAcc. These conditional endpoints are not prevalence or an overall controller-safety estimate.

GLM. Under the strict actionable-core conditions, refreshed-winner writes occur in 8/8 Qwen and 6/8 GLM opportunities, versus 0/4 Stable controls for each model. In one trace, Qwen records REM-1A before refresh and then sends REM-1B to the mutation tool. The remaining five Qwen and two GLM wrong writes arise in the fallback-policy slice. These trajectories locate the failure between binding and action; the fixed replay only verifies target-to-write consistency.

Decision Visibility and Transfer

The Evidence Map records experimental chronology; this section uses the shorter dataset names. The full-diagnostic matched-call test compares History-only and Decision-visible actors with equal calls, base payloads, states, and tool schemas. The actor-visible decision block is the manipulated field under matched calls and base payloads; the compiler representation remains part of the probe.

Authored diagnostic. On 32 actionable changed pairs, PairAcc rises from 5/32 to 13/32 for Qwen (+25.0 points [6.3,46.2]) and from 8/32 to 25/32 for GLM (+53.1 [28.6,77.8]). Actionable E2E changes from 100/128 to 106/128 and from 102/128 to 120/128. Substitution among correctly bound actionable Preserve cases falls from 22/28 to 13/28 and from 16/25 to 0/25. All 320 task-model rows complete, comprising 640 actor outputs and 320 compiler outputs, without API or parse failure.

Offline Decision-enforced raises Qwen PairAcc to 24/32, with 18 repairs and eight row-level harms. GLM remains at 25/32; enforcement repairs four Reject rows and harms none. The visibility result is consistent across the two authored

TRI SUBSTITUTIONS DOMINATE GENERIC WRONG WRITES

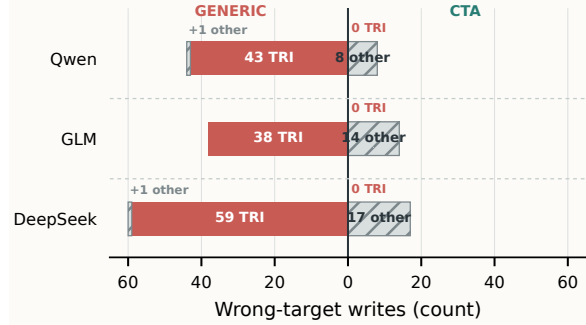


Figure 4: Wrong-target writes under fixed-executor replay of the 240-task Cross-Schema Replication. Generic is shown left and CTA right; coral segments are conditional TRI substitution writes and hatched segments are other wrong writes. Generic produces 43/38/59 TRI writes plus two non-core errors. CTA records zero TRI writes; its 8/14/17 errors arise from binding, grounding, or execution failures.

models, while hard enforcement has mixed effects.

Human rewrites. Against 48 determinate rewrite majorities, CTA reaches 89.6% for Qwen and 93.8% for GLM. In the matched-call transfer, Qwen actionable E2E remains 30/40 and majority accuracy changes from 34/48 to 35/48. GLM improves from 31/40 to 39/40 and from 33/48 to 41/48. Only three actionable changed pairs are complete: Qwen changes from 0/3 to 1/3 and GLM from 1/3 to 3/3, with wide cluster intervals. These one-volunteer rewrites test authored task semantics, not open-language validation.

Source-derived pairs. The three-source contrast contains 30 changed-winner pairs grounded in STATE-Bench, Agent-Dojo, and ToolSandbox states and tool schemas. PairAcc changes from 12/30 to 13/30 for Qwen, 11/30 to 20/30 for GLM, and 19/30 to 22/30 for DeepSeek. Actionable E2E changes from 39/60 to 39/60, 37/60 to 48/60, and 45/60 to 47/60. Only GLM has an E2E interval excluding zero (+18.3 points [8.3,30.0]); Qwen is null and the DeepSeek interval crosses zero. Preserve substitution is unchanged at 7/26 for Qwen, but falls from 10/30 to 1/30 for GLM and from 6/27 to 2/27 for DeepSeek. All wrong-target writes change from 16 to 17, 16 to 5, and 11 to 9. Source-specific slices are supplementary; their heterogeneity is retained rather than pooled into a universal transfer claim. These controlled interventions over source-derived states and schemas connect the controlled diagnostic to source interfaces; the public-suite audit below examines native opportunity coverage separately.

Figure 5 summarizes the matched-call effects across authored, rewritten, and source-derived tasks. The contrast helps both models on the authored PairAcc endpoint, but transfer is model-dependent.

Alternative operationalizations. The primary Qwen package comparison and GLM replication are call-asymmetric mechanism probes. Lifecycle-Gated improves over Generic on both, while CTA and a full-history timing

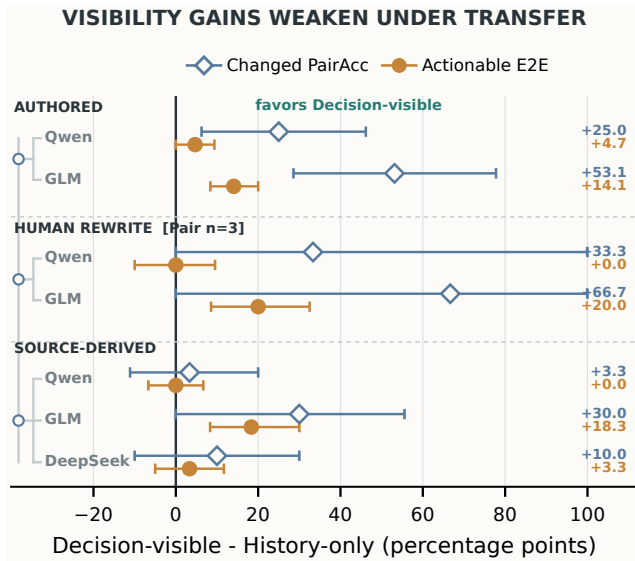


Figure 5: Decision-visible minus History-only effects under equal calls and matched base payloads. Diamonds show changed-winner PairAcc and circles show actionable E2E differences; bars are cluster-bootstrap 95% intervals. The branching labels organize authored, human-rewrite, and source-derived evidence; human-rewrite PairAcc contains only three complete actionable pairs. Authored effects favor visibility for both models, whereas only the GLM source-derived E2E interval excludes zero.

reminder show that lifecycle typing and hard gating are not unique implementations. Full results and intervals are supplementary.

The initial no-LLM event-order rule scores 60.6% on Matched Timing Diagnostic. Rule*, written after error inspection, reaches 92.5% on Matched Timing Diagnostic, 96.0% on rewrites, and 91.7% on Cross-Schema Replication. This strong authored-inventory result shows that the diagnostic contains usable event-order cues. On the source-derived pairs, the frozen rule reaches 15/60 exact targets and 2/30 PairAcc; timestamps, extra numeric fields, and unfamiliar ranking expressions expose parser and selector-grounding errors. The residual and component analyses are supplementary. Together they characterize typing, compilation, reminders, rules, and gates as alternative implementations of the same timing decision.

Persistence-policy adaptations. We author-adapt the Binding Drift entity-lock semantics and self-reverification prompt to all 240 actionable cross-schema tasks (Babu and Indukuri 2026a). Entity lock favors Preserve and self-reverification favors Reevaluate, characterizing the two persistence policies. The adapted verifier receives the instruction and refreshed candidates without S_0 or the pre-refresh bound ID, so this is not an official reproduction or an information-matched performance comparison. Detailed mode and pair counts are supplementary.

Boundary analyses. External evidence is mixed. A 24-task ToolSandbox-style pilot finds GLM substitution in 3/6 eligible changed-winner cases versus 0/2 Stable cases, while Qwen is 0/6. Its four-condition, 96-task extension finds zero conditional substitutions in 64–87 eligible opportunities. Across 20 STATE-Bench and AgentDojo clusters (Microsoft Research 2026; Debenedetti et al. 2024), Qwen ordinary history substitutes in 2/7 eligible AgentDojo Preserve/Changed rows versus 0/7 matched Stable rows; the other repository-model cells are zero and execution accuracy has no consistent winner.

Under an author-built checklist and retrieval procedure, the public-suite audit identifies no strict native opportunity in pinned ToolSandbox (129 families), AppWorld (244 families), or τ^3 -Bench (2,449 tasks; 10,832 trajectories). A structural scan of API-Bank, BFCL, and ToolTalk also finds zero. The checklist accepts all 30 injected strict positives and excludes all 30 one-condition-missing hard negatives, but natural retrieval recall has not been independently calibrated.

Structural stress tests remain mixed. A two-refresh Qwen test favors Generic over scalar Lifecycle (32/40 vs. 28/40), while held-out role indexing improves 35/40 to 39/40. GLM role ITT is 37/40 versus 40/40 scalar because of three transport failures and ties after recovery. These results set the current scope at single-refresh scalar mutations.

Discussion

The evidence chain starts with a correct observable binding and ends with a target-level state change. Generic often carries the initial ID through refresh, recomputes its selector on S_1 , and sends the new winner to the mutation tool. PairAcc exposes this selective-policy failure, while conditional substitution distinguishes it from initial binding and grounding errors. Fixed replay checks whether the selected target produces the corresponding write; SQLite trajectories provide the model-facing execution evidence.

CTA, lifecycle records, timing reminders, and deterministic rules all operationalize a resolution-timing decision. Their relative performance supports no unique controller design. Rule* is especially informative: event order plus benchmark-aware lexical coverage is sufficient for high accuracy on the authored inventories, while source-derived ranking expressions introduce parser and selector-grounding failures. The benchmark can therefore be solved by a simple executable rule when the expression inventory is covered; open wording adds a separate implementation challenge.

Evidence scope. The Evidence Map separates the primary Qwen package run from replications, matched-call tests, audits, and post-hoc analyses. The primary package comparison measures an end-to-end controller contrast. The equal-call experiments isolate visibility of the timing decision, and the rule and component studies examine alternative implementations. This organization supports a diagnostic claim about selective re-resolution, with architecture ranking outside its scope.

External evidence is mixed. Eligible low-intervention runs show no conditional substitutions, the audited suites contain no strict native cases, and retrieval recall has not been in-

independently calibrated. The source-derived matched pairs supply controlled transfer evidence, with a clear E2E gain only for GLM. A broader benchmark will need observable initial bindings, opposite-gold transitions, and target-level execution records in naturally occurring workflows.

Limitations

The controlled diagnostic uses single-refresh scalar tasks and simplifies ambiguity, repair, partial observability, identity migration, and composition. Cross-Schema Replication varies schemas and states while retaining authored language templates. Volunteer rewrites preserve authored task semantics, and only three actionable changed pairs are complete. A model-authored language audit fails its semantic gate because two judges accept no complete pair. Open-language generalization therefore remains unresolved.

Referent identity is better supported than fallback policy: agreement is weak when the old target is action-invalid, and Reject, Clarify, and authorized reselection may all be reasonable execution choices. Multiple refreshes and referent roles also require richer control state; the two-refresh results show that the scalar record does not compose automatically.

The public-suite and low-intervention results may reflect low prevalence, benchmark coverage, or retrieval misses. Current evidence cannot separate these explanations. The primary package comparison is call-asymmetric, whereas the equal-call studies supply the initial ID and isolate decision visibility. Temperature-zero repeats vary at task level, and provider weights lack immutable revision identifiers.

Conclusion

TRI makes selective referent re-resolution identifiable by pairing the same transition with bound and deferred action references. Changed-winner PairAcc rejects both unconditional target policies, and the execution trace connects substitution after correct binding to wrong-entity writes. Exposing an explicit timing decision helps Qwen and GLM on the authored matched-call test; transfer is model-dependent, with a clear source-derived E2E gain only for GLM. Rule*, hard enforcement, and composition results show that the same control decision can be represented in several ways and limit the algorithmic novelty claim. The current evidence covers controlled single-refresh scalar workflows. Native-workflow frequency and general runtime behavior remain unmeasured; open-language coverage remains open.

References

Babu, R. S.; and Indukuri, S. 2026a. Binding Drift in Multi-Step Tool-Augmented Agents. Concurrent preprint, arXiv:2607.18316.

Babu, R. S.; and Indukuri, S. 2026b. Entity Binding Failures in Tool-Augmented Agents. arXiv:2606.30531.

Barres, V.; Dong, H.; Ray, S.; Si, X.; and Narasimhan, K. 2025. τ^2 -Bench: Evaluating Conversational Agents in a Dual-Control Environment. arXiv:2506.07982.

Beurer-Kellner, L.; Fischer, M.; and Vechev, M. 2023. Prompting Is Programming: A Query Language for Large Language Models. In *Proceedings of the 44th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 1946–1965.

Chen, Z.; Liu, H.; Xu, D.; Dong, D.; Li, J.; Pu, B.; and Zhai, J. 2026. Cordon: Semantic Transactions for Tool-Using LLM Agents. arXiv:2606.17573.

Cheng, Y.; Moakhar, A. S.; Fan, C.; Hosseini, P.; Faghieh, K.; Sodagar, Z.; Wang, W.; and Feizi, S. 2026. Your LLM Agents Are Temporally Blind: The Misalignment Between Tool Use Decisions and Human Time Perception. Findings of ACL 2026, arXiv:2510.23853.

Dasigi, P.; Liu, N. F.; Marasović, A.; Smith, N. A.; and Gardner, M. 2019. Quoref: A Reading Comprehension Dataset with Questions Requiring Coreferential Reasoning. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 5925–5932.

Debenedetti, E.; Zhang, J.; Balunović, M.; Beurer-Kellner, L.; Fischer, M.; and Tramèr, F. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*.

Gardner, M.; Artzi, Y.; Basmova, V.; Berant, J.; Bogin, B.; Chen, S.; Dasigi, P.; Dua, D.; et al. 2020. Evaluating Models’ Local Decision Boundaries via Contrast Sets. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 1307–1323.

Grosz, B. J.; Joshi, A. K.; and Weinstein, S. 1995. Centering: A Framework for Modeling the Local Coherence of Discourse. *Computational Linguistics*, 21(2): 203–225.

Ji, H.; Xiong, K.; Han, S.; Xia, P.; Qiu, S.; Zhou, Y.; Liu, J.; Li, J.; Li, B.; Zheng, Z.; Xie, C.; and Yao, H. 2026. ClawArena: Benchmarking AI Agents in Evolving Information Environments. arXiv:2604.04202.

Joshi, M.; Chen, D.; Liu, Y.; Weld, D. S.; Zettlemoyer, L.; and Levy, O. 2020. SpanBERT: Improving Pre-training by Representing and Predicting Spans. *Transactions of the Association for Computational Linguistics*, 8: 64–77.

Kamp, H.; and Reyle, U. 1993. *From Discourse to Logic: Introduction to Model-Theoretic Semantics of Natural Language, Formal Logic and Discourse Representation Theory*. Kluwer Academic Publishers.

Khattab, O.; Singhvi, A.; Maheshwari, P.; Zhang, Z.; Santhanam, K.; Vardhamanan, S.; Haq, S.; Sharma, A.; et al. 2024. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. In *International Conference on Learning Representations*.

Lee, K.; He, L.; Lewis, M.; and Zettlemoyer, L. 2017. End-to-end Neural Coreference Resolution. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, 188–197.

Li, M.; Zhao, Y.; Yu, B.; Song, F.; Li, H.; Yu, H.; Li, Z.; Huang, F.; and Li, Y. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 3102–3116.

Li, Y.; Chen, Y.; Wen, H.; Zhang, B.; Liu, H.; Wang, P.; Feng, Y.; and Tian, Y. 2026. VIGIL: Runtime Enforcement of Behavioral Specifications in AI Agent Skills. arXiv:2606.26524.

Liu, G.; and Gabriel, S. 2026. PM-Bench: Evaluating Prospective Memory in LLM Agents. In *Conference on Language Modeling*. ArXiv:2607.12385.

Liu, X.; Yu, H.; Zhang, H.; Xu, Y.; Lei, X.; Lai, H.; Gu, Y.; Ding, H.; Men, K.; et al. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations*.

Lu, J.; Holleis, T.; Zhang, Y.; Aumayer, B.; Nan, F.; Bai, H.; Ma, S.; Ma, S.; Li, M.; Yin, G.; Wang, Z.; and Pang, R. 2025. ToolSandbox:

- A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 1160–1183.
- Ma, B.; and Miyao, Y. 2026. The Imperfective Paradox in Large Language Models. arXiv:2601.09373.
- Microsoft Research. 2026. STATE-Bench: Benchmark for Enterprise Workflows. GitHub software release. Pinned commit 0962c71.
- Mondl, A.; Maisel, M.; and Brock, J. H. 2026. Autoformalization of Agent Instructions into Policy-as-Code. arXiv:2606.26649.
- Packer, C.; Wooders, S.; Lin, K.; Fang, V.; Patil, S. G.; Stoica, I.; and Gonzalez, J. E. 2023. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560.
- Patil, S. G.; Zhang, T.; Wang, X.; and Gonzalez, J. E. 2023. Gorilla: Large Language Model Connected with Massive APIs. arXiv:2305.15334.
- Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; et al. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *International Conference on Learning Representations*.
- Ribeiro, M. T.; Wu, T.; Guestrin, C.; and Singh, S. 2020. Beyond Accuracy: Behavioral Testing of NLP Models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 4902–4912.
- Ruan, Y.; Dong, H.; Wang, A.; Pitis, S.; Zhou, Y.; Ba, J.; Dubois, Y.; Maddison, C. J.; and Hashimoto, T. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *International Conference on Learning Representations*.
- Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*.
- Sierra Research. 2026. τ^3 -Bench 1.0.0: Voice, Knowledge, and Task Quality. GitHub software release.
- Sohail, S.; and Haider, G. 2026. Bounded Autonomy for Enterprise AI: Typed Action Contracts and Consumer-Side Execution. arXiv:2604.14723.
- Sumers, T. R.; Yao, S.; Narasimhan, K.; and Griffiths, T. L. 2024. Cognitive Architectures for Language Agents. *Transactions on Machine Learning Research*.
- Tang, Z.; Zhao, Q.; Franco, G.; Wijaya, D.; Mueller, A.; Schuster, S.; and Kim, N. 2026. Do Language Models Track Entities Across State Changes? In *International Conference on Machine Learning*. ArXiv:2605.30233.
- Trivedi, H.; Khot, T.; Hartmann, M.; Manku, R.; Li, V. D.; Gupta, S.; Sabharwal, A.; and Balasubramanian, N. 2024. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. arXiv:2407.18901.
- Uddin, M. N.; Saeidi, A.; Blanco, E.; and Baral, C. 2026. Ledger-Agent: Structured State for Policy-Adherent Tool-Calling Agents. arXiv:2606.20529.
- Wang, H.; Poskitt, C. M.; and Sun, J. 2025. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. arXiv:2503.18666.
- Wang, Z.; Wu, W.; Wang, G.; Ye, G.; and Cheng, Z. 2026. MetaEval: Measuring the Discrimination of Benchmarks for Efficient LLM Evaluation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 33773–33781.
- Willard, B. T.; and Louf, R. 2023. Efficient Guided Generation for Large Language Models. arXiv:2307.09702.
- Xu, J.; Li, Q.; Wu, J.; Lan, Y.; Li, S.; Zhou, H.; Jiang, B.; Wang, L.; Wang, J.; Luu, A. T.; Xiong, C.; Park, H. W.; Hooi, B.; and Hu, Z. 2026. EvoArena: Tracking Memory Evolution for Robust LLM Agents in Dynamic Environments. arXiv:2606.13681.
- Yang, L.; Wang, Z.; Tang, X.; Zhou, S.; Chen, D.; Jiang, W.; and Li, Y. 2026. ProBench: Benchmarking GUI Agents with Accurate Process Information. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 27547–27555.
- Yao, S.; Shinn, N.; Razavi, P.; and Narasimhan, K. 2024. Tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045.